

武汉大学

本科毕业论文（设计）

面向任意形状数据的快速聚类算法研究

姓 名：	杨 永 豪
学 号：	2020302111192
专 业：	软件工程
学 院：	计算机学院
指导教师：	黄浩 教授

二〇二四年 四 月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：

指导教师签名：

日 期： 年 月 日

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：

指导教师签名：

日 期： 年 月 日

摘 要

任意形状聚类是数据挖掘领域的一项开放性挑战，其目的是直接对未经处理的数据集进行划分，将其归类为若干具有区分度的不同聚类。纵观当今整个计算机市场，任意形状的聚类模型在计算机图像的分割、市场行为研究分析等多个学科领域均有大量应用。尽管已有一些高效的聚类算法，但对更快速、更准确、适用性更强的算法的研发需求仍然一直存在。相较于其他聚类方法而言，任意形状聚类不对数据集的分布作出预先假设，因而适用面更广，但与此同时也导致了该类算法的复杂度普遍较高，会消耗更多的算力和时间资源。为了解决这一问题，部分已有算法提出通过减小数据集规模来减少时间消耗。然而，不合理的数据点选取往往会破坏数据的原有结构，最终导致算法的聚类能力下降。因此，本文提出了一种基于形状保持的任意形状快速聚类算法。该算法首先使用形状保持算法在数据集中均匀取点，之后对取得的点进行调整以使聚类结构更为清晰，最后利用基于类间最小距离的凝聚算法得到聚类结果。本文提出的算法在人工数据集和真实的数据集上进行了大规模的实验，实验结果证明了本方法相比较于其它主流的聚类算法而言是具有较好的适用性的。

关键词： 任意形状聚类；形状保持采样；凝聚聚类

ABSTRACT

The arbitrary shape clustering is an open challenge in the field of data mining, and the purpose is to directly classify unprocessed datasets into a number of different clusters with degrees of discrimination. Throughout the whole market, arbitrary shape clustering has been applied in image segmentation, market research and many other fields, while the demand for faster, more accurate and more applicable clustering algorithms has always existed. Compared with other clustering methods, arbitrary shape clustering does not make prior assumptions about the distribution of the data set, so it is more applicable. But at the same time, the complexity of this algorithm is generally higher, which will consume more computing power and time resources. To solve this problem, some existing algorithms have proposed to reduce time consumption by reducing the size of data sets. However, unreasonable data point selection often destroys the original structure of the data, which eventually leads to the clustering ability of the algorithm. Therefore, this paper proposes a fast clustering algorithm for arbitrary shape based on shape preserving sampling. The algorithm first uses The shape preserving sampling to take points evenly in the data set, then adjusts the obtained points to make the clustering structure more clear, and finally obtains the clustering result using the agglomerative clustering algorithm based on the representative points of adjacent boundaries. The algorithm in this paper conducts large-scale detection on artificial and real datasets, and it is experimentally proved to Have better clustering ability and faster running speed compared with general algorithms.

Key words: arbitrary shaped clustering; shape preserving sampling; agglomerative clustering

目录

摘 要	I
ABSTRACT	II
1 绪论	1
1.1 研究背景和意义	1
1.2 研究现状	2
1.3 主要工作	2
1.4 论文组织结构	3
2 相关工作	4
2.1 任意形状聚类方法	4
2.1.1 层次聚类方法	4
2.1.2 基于图的方法	5
2.1.3 基于密度的方法	6
2.1.4 基于划分的方法	8
2.1.5 基于采样的方法	9
2.2 相似度度量	9
2.2.1 闵可夫斯基距离	9
2.2.2 余弦相似度	10
2.2.3 正交标准化	10
2.3 性能度量	11
2.3.1 外部指标	11
2.3.2 内部指标	13
3 PHANTOM 算法	15
3.1 形状保持采样	16
3.2 代表点位置调整	18
3.3 基于类间最小距离的凝聚聚类	20
3.4 算法复杂度分析	21
3.5 本章小结	22

4	实验分析	23
4.1	实验设置	23
4.2	参数选择	24
4.2.1	代表点个数	24
4.2.2	k 近邻参数	24
4.2.3	迭代次数	25
4.3	聚类能力验证	26
4.4	算法性能对比	27
4.5	算法可拓展性对比	28
5	总结	30
	参考资料	31
	致谢	34

1 绪论

1.1 研究背景和意义

近年来,伴随着新一代计算机科学技术研究突破性的快速发展以及新一代互联网应用的大量出现,“大数据”这一新词概念逐渐开始步入公众视野。人们日常生活中的衣食住行无时无刻不在这个世界上留下数据的痕迹,而硬件的升级和云平台的兴起使得存储这些数据成为可能。然而,海量的数据却并不都能在短期内直接内化为可用于实际生产的知识资源,只有经过大量科学分析整理的数据资料才能变为有价值的有用信息。为了深度挖掘各种隐藏在庞大数据资源中鲜为人知的真正有用的价值信息,数据挖掘问题成为了整个人工智能理论和计算机数据库领域内研究中的热点问题。数据挖掘是一项通过综合分析研究大量具体数据,从这些具体数据信息中来寻找其存在规律进而对其进行预测的一种技术,而聚类算法正是其中重要的组成部分。

聚类分析(又称群分析)是一种由分类问题衍生出的统计分析方法,其核心是合并相似的样本,区分不同的样本,并据此再对样本进行聚类。聚类分析的特点是聚类信息全部来自数据点本身,待划分的类是未知的,并且待分类的数据点个体之间不具有差异性,仅能根据其位置分布和疏密程度寻找最合理的分类。因此,聚类分析又被称为无监督分类(即没有先验知识的结果作为监督)。

在实践中,为数据集选择合适的聚类方法依赖于数据集中聚类的数据模式。例如,共聚类和子空间聚类更适合处理仅在部分特征上可区分的聚类,罕见类别聚类和少数检测更适合寻找罕见类别的聚类[1]。本文主要关注那些能够挖掘任意形状聚类的聚类算法。

任意形状聚类算法是一类十分特殊的聚类算法,其对于数据样本的划分完全基于数据点的相似度,同时对数据分布的形状不做要求,即在形状上具有“任意性”。随着现代社会的发展以及大数据云计算等行业的兴起,对数据的分析和处理的需求也与日俱增。任意形状聚类在数据挖掘[2]、图像处理[3]、自然语言处理[4]、生物信息学[5-6]和物联网等诸多领域均发挥着重要的作用。例如,聚类算法可用于数据挖掘中的用户行为分析、产品定位和市场分析[7-8]等;在图像处理上,任意形状聚类在图像分割算法、目标检测以及图像压缩等方向上都发挥着重要作

用；不仅如此，任意形状聚类还可以用于新闻分类、情感分析、手写识别和文本推荐等自然语言处理；除此之外，生物信息学中的基因表达数据的聚类和分类以及物联网传感器网络中的数据处理都离不开任意形状聚类算法的支持。可以说，任意形状聚类算法在诸多行业有着良好的应用前景，同时市场的快速发展也对算法提出了越来越高的要求。

1.2 研究现状

任意形状聚类问题[1]，是指在有些时候，数据集中存在的数据样本所蕴含的聚类结构是更加复杂、不规则的形状，而不是简单的球形。这种不规则的数据聚类通常被称为任意形状聚类。任意形状聚类算法的目标是快速准确地从数据集中识别出这些不规则的数据聚类。虽然很多传统的聚类方法，例如K均值算法等，可以高效地识别出数据分布呈球形或凸形状的数据聚类。但是，当这些聚类具有不规则形状(如凹形)的数据簇时，聚类结果通常会出现与数据聚类的实际分布不匹配的情况。

对任意形状聚类问题的研究由来已久。从最基础的基于密度聚类[9-10]、层次聚类[11]和基于采样聚类[12-13]等方法上已经衍生出大量的变种方法。任意形状聚类算法的优点在于可以自动发现数据的内在结构和规律，可以适用于数据挖掘、图像处理、自然语言处理、生物信息学和物联网等难以提前判断数据特征的领域。然而，不限制数据集的缺点也很明显，那就是时间和计算资源的浪费难以避免。就目前而言，主流的大多数任意形状聚类算法都容易受到噪声数据和异常值的影响[14]，同时聚类结果也需要经过人工分析和解释才能变得直观，对使用者的专业素养要求高，现阶段因其门槛高，限制大而不能在市场中完全普及。为了解决这一问题，部分已有算法[12,13,15,16]提出通过减小数据集规模来减少时间消耗。然而，不合理的数据点选取往往会破坏数据的原有结构，最终导致算法的聚类能力下降。

1.3 主要工作

为了解决目前已有任意形状快速聚类方法在针对大规模数据集时普遍存在的高复杂度和易受噪点影响的问题，本文提出了一种基于采样的 PHANTOM 算法。

该算法先通过形状保持采样在原始数据集中获取代表点，再通过调整代表点使其聚类结构更清晰，接着利用基于类间最小距离的凝聚聚类完成对代表点的分类，最后依照代表点的分类完成对其余原始数据点的分类。为了验证本文算法在时间和分类能力上较其他主流算法的优势，本文在人工数据集和真实数据集上对PHANTOM算法的性能进行了对比验证，通过实验证明了其在性能上和可拓展性上要优于其他主流任意形状聚类算法。

1.4 论文组织结构

本文分析和研究了目前主流的任意形状聚类算法的优势和不足，并提出一种能实现任意形状快速聚类的PHANTOM算法。本文总共分为五个章节，每章节的安排如下：

第一章为绪论，简单阐述了围绕本章课题研究展开的基本研究理论的历史背景和研究发展现状分析。

第二章为相关基础性工作，主要系统阐述了国内外目前几类主流的任意形状聚类的方法理论及其局限性和不足之处，同时重点给出了如何评价这些算法性能的其他相关指标。

第三章为算法讲解，详细论述了本文提出的PHANTOM算法实现任意形状快速聚类的详细步骤，再基于此给出了本文算法的复杂度分析。

第四章为实验分析，包含了算法参数的具体设置以及性能验证和对比，直观展示了该算法的实用性和可拓展性。

第五章为总结部分，概括了该算法的优势与不足，并对未来的工作进行了展望。

2 相关工作

2.1 任意形状聚类方法

对任意形状聚类算法的研究目前可以分为以下几种：层次聚类方法、基于图的方法、基于密度的方法和基于采样的方法。

2.1.1 层次聚类方法

层次聚类是一种经典的任意形状聚类方法，其目标是将数据集划分为具有明显区分度的不同层级，最后得到结构为树状的聚类结果[11]。从原理上说，层次聚类在不同的“层次”上对样本数据进行划分，并据此一层层地对数据进行聚类。在聚类树中，不同类别的原始数据点被看作是树的最底层，而聚类的根节点则被看做是树的最顶层。这种方法由此自顶向下或自底向上地将每一个样本进行合并聚集，最终得到嵌套的簇和类似树的层次结构。总体而言，层次聚类的聚类任务往往是通过生成一系列嵌套的聚类树来完成，并在不同层次对数据进行划分来揭示其内在的结构关系。

例如，凝聚聚类 (agglomerative clustering) 就是一种比较基础的层次聚类方法。它的实现非常简单。首先，凝聚聚类将每一个数据点都视为一个单独的数据簇 (也就是上文提到的根节点)；接着，该方法通过某些准则找到最为相似的数据簇并将其归为一类，如此重复直到剩余数据簇的数量降到给定值，这时得到的不同数据簇即为我们需要的分类结果。

根据合并准则的不同，凝聚聚类又有不同的实现方式。常见的凝聚聚类判别方法如下：

1. 方差判别法：该方法在每次合并数据簇的时候需要确保聚类中所有数据簇的方差增加量最小，最终结果通常为若干大小差不多的数据簇 (数据簇数量不均会大幅增加整体的方差)；

2. 中点判别法：在每次合并簇时总选取簇中点之间最大距离最小的两个簇进行合并；

3. 平均判别法：在每次合并簇时总选取满足簇 A 中的所有点到簇 B 中的所有点的距离平均值最小的两个簇进行合并；

4. 类间最小距离判别法：在每次合并簇时总选取不同簇中距离最近的数据点间的距离来代表簇间距离，再根据该距离最小值对应的两个数据簇进行合并。本文即是使用了该方法进行代表点间的聚类。

2.1.2 基于图的方法

基于图的图像聚类算法将图像看作一个由像素点和像素点之间的关系组成的图，然后使用图论算法对图像进行聚类。其中，最常用的算法是基于最小生成树的聚类算法。这种算法将图像看作一个带权无向图，将像素点看作节点，将像素点之间的距离作为边的权重，然后使用最小生成树算法对图像进行聚类。除此之外，基于图的聚类算法还有谱聚类算法 (Spectral Clustering)，社区发现算法 (Community Detection) 和变色龙算法 (Chameleon) 等。

谱聚类算法将数据点看作图上的节点，通过计算节点之间的相似度矩阵和拉普拉斯矩阵来确定节点之间的相似性和距离，然后通过对拉普拉斯矩阵进行特征分解来得到聚类簇的划分。其中，根据谱聚类的定义[17-18]，谱聚类的输入是从数据样本的特征向量计算得出的数据样本的相似性矩阵，以表示数据集中样本之间的相似性关系。

社区发现算法将数据点看作社交网络上的节点，通过计算节点之间的社交关系和相似度来确定节点之间的相似性，然后利用社区发现算法将相似度高的节点划分到同一个社区中。在提升社区发现算法性能上已有不少尝试。例如，Tiantian Zhang[19] 等人尝试通过寻找核心节点来“以偏概全”，强化算法的局部性能，Qinna Wang[20] 等人则尝试通过模糊化处理不确定节点来提高算法可信度。

Chameleon 算法[21]是一种基于图的两阶段层次聚类算法，主要优势在于其在识别最相似的集群时同时考虑了数据簇间的互连性和紧密性。Chameleon使用两阶段算法在数据集中查找聚类：在第一阶段，Chameleon 使用图分区算法将数据项聚类到几个相对较小的子聚类中；在第二阶段，它使用基于凝聚的层次聚类算法，通过重复组合上一阶段得到的子集群来找到真正的集群。

2.1.3 基于密度的方法

基于密度的聚类算法是一类备受瞩目的任意形状聚类算法，原因在于其不需要对聚类个数做出预先假设，分类的结果完全基于数据点在空间中的密度分布。其核心思想是通过样本点邻域内（邻域大小通过输入参数 ϵ 控制）的邻居数（最小邻居数通过输入参数 $MinPts$ 控制）定义样本空间的密度。算法的流程是依次计算每一点处的密度值，再根据得到的密度来定义簇。该算法的局限性在于只能在聚类空间中的区域内所包含对象（点或其他空间对象）的数目不小于某一给定阈值时才可使用，即只能在数据点密度足够的空间中使用。

DBSCAN(Density-Based Spatial Clustering of Applications with Noise) 聚类算法是一种主流的基于密度的聚类算法，它可以根据数据点的密度分布识别出任意形状的聚类簇，并且对噪声数据有很强的抗干扰能力。DBSCAN 算法的基本思想是通过寻找数据点周围的相邻点再以这些相邻点为核心向外延伸，最后判断寻找到的数据点是否应该被分类到同一个簇中。DBSCAN 算法需要输入两个参数：样本点邻域半径 ϵ 和样本点邻域内最小的邻居个数 $MinPts$ 。其中，前者定义了数据点间的邻域范围，即距离核心点不超过邻域半径的数据点都被认定为是核心点的邻居；后者则定义了一个簇的最小密度，即在一个数据点的邻域范围内，如果包含的数据点个数不少于最小邻居数，则认为该点是核心点，可以以其为中心构成一个簇。DBSCAN 的实现过程由以下几步构成：

1. 随机选取一个还没有被算法遍历访问到的数据点 p ；
2. 计算数据点 p 的一个较大邻域范围内所有数据点到其的距离，如果得到的距离不超过预先输入的半径 ϵ ，则认为这些点属于同类，将它们合并到同一个簇中；
3. 对于簇中的每个数据点，继续递归地执行步骤 2，直到该簇内的每个数据点都已经被遍历且不再有新的点加入该簇为止；
4. 重复执行步骤 1 到 3，直到所有数据点都至少被访问过一次为止。

DBSCAN 的优点：

1. 相较于K-Means 之类的只适用于凸数据集的聚类算法，DBSCAN可以对任意形状的稠密数据集进行聚类；
2. 可以在完成聚类任务的同时发现噪点，对噪点不敏感，聚类结果受噪点的影响小，抗干扰能力优秀；
3. 聚类结果随机性小，不偏倚，只会因输入参数的不同而发生改变。相对的，K-Means 之类的聚类算法初始值对聚类结果有很大影响。

DBSCAN 的缺点：

1. 如果样本集的密度分布差异大、聚类间距差相差很大时，聚类质量较差[9]，这时不宜使用 DBSCAN，而对于稀疏分布的数据集，DBSCAN更是无法建立基础的数据簇，此时不能选用DBSCAN；
2. 对于较大的样本集，DBSCAN的聚类收敛时间较长[10]（因为其时间复杂度为 $O(N^3)$ ），因此要采用其他方法进行改善[10]。
3. DBSCAN的参数相较于其他传统聚类算法而言更难调整，因为邻域半径和相邻点个数与分类结果间缺乏对应关系。相较于其他算法直接指定聚类个数，这些参数对结果的影响是不直观的，且不同的参数组合对最后的聚类效果有较大影响。

为了改善 DBSCAN 的性能，解决以上缺点，目前已有多种基于 DBSCAN 的变种方法。例如，Negar Ohadi 等人[9]提出了一种基于网格划分的SW-DBSCAN 算法，根据数据点密度的不同调整网格的尺寸和形状，能够有效应对样本集的密度不均匀问题；又例如，P. Viswanath 等人[10]提出了一种基于快速混合密度的l-DBSCAN 算法，该算法同时使用两种类型的原型，一种是用于减少时间要求的较粗的原型，另一种是用于减少结果偏差的更精细的原型。两者相结合能使 DBSCAN算法能够处理大规模数据集，并使其时间消耗降至可接受的范围。

2.1.4 基于划分的方法

基于划分的聚类方法又被称为基于分区的聚类方法或基于距离的聚类方法，其基本目标是将原始数据集分为 k 个聚类，同时保证分类得到的各聚类中的样本间距离最小化。以K-Means算法为例，其通过不断迭代将数据集分为 K 个簇（其中 K 是给定的参数），使得各个数据簇中的数据点到各自簇中心的距离之和最小。K-Means算法在每一次迭代的过程中会将每一个数据点分到离它最近的簇中心，并重新计算更新后的各簇的中心位置，直到每个数据点都不再改变类别或K-Means算法达到最大迭代次数为止。

步骤:

1. 在原始数据集中随机的选取 K 个中心点，并让它们各自代表最初的 K 个类别；
2. 计算原始数据集中的所有样本点到 K 个中心点之间的欧氏距离；
3. 将每个样本点划分到与其距离最小的中心点类别中；
4. 计算每个类别中样本点的均值，得到 K 个均值，将这 K 个均值作为新的中心点（ K 均值这一名称的由来）；
5. 重复步骤2、3、4直到各数据点的分类不再更新或迭代次数达到上限；

执行完以上步骤后，由K-means算法得到了所有数据点的 K 个分类。

K-Means算法的评分函数由下式给出：

$$\sum_{m=1}^k \sum_{t_{mi} \in K_m} (C_m - t_{mi})^2 \quad (2.1)$$

其中 k 表示预先输入的需要分类的聚类总个数， t_{mi} 表示每个待分类的数据对象， C_m 表示每次迭代时各聚类的中心点，K-Means算法的核心即是通过迭代调整使得函数 (2.1) 的值最小，并且该值不再随着迭代而改变，此时得到的各点分类就是 k 均值的分类结果。

2.1.5 基于采样的方法

采样一直是数据挖掘领域的重要课题。为了减小数据集规模以加快聚类的速度，对数据集进行采样就显得尤为重要，而不同的方法则会给结果带来不同的影响。例如，随机采样的速度较快，但是得到的数据集并不平均，且采样结果不稳定，容易破坏数据的结构特征。与之相对的，行列式点过程能够得到更为平均的数据点，但它的运行速度过慢，容易浪费时间和计算资源。因此，本文提出的算法最终选取了形状保持采样方法来处理数据集。

采样不仅能够对数据集进行预处理，还能直接用于聚类。例如，SPARCL[12] 和 ABACUS[13] 都使用了基于聚类的采样方法，使代表点向聚类中心靠拢，增加代表点的代表性。然而，原始数据簇的局部密度较大时以上方法会使代表点的分布被扭曲，进而降低了聚类形状的可辨识度。为了解决这一问题，CLASP 算法[1] 提出了基于网格的采样方法，通过网格划分出高密度的代表点区域，缺点则是网格设置不合理时可能造成低密度区域被舍弃，最终影响到聚类结果。

2.2 相似度度量

相似度度量 (Similarity Measurement) 用于估算不同样本之间的相似程度，常作为分类问题的判断标准。机器学习和数据挖掘中常需要知道个体间差异的大小，进而评价个体的相似性和类别，因此引入相似度度量就非常有必要。评价样本间相似度常用的方法是距离计算和余弦相似度计算，若样本间的距离越小，则相似度越高；若样本间的余弦相似度越大，则相似度越高。

2.2.1 闵可夫斯基距离

闵可夫斯基距离 (Minkowski distance) 的计算方法由下式给出：

$$dist(x_i, x_j) = (\sum_{u=1}^N |x_{iu} - x_{ju}|^p)^{1/p} \quad (2.2)$$

其中当 $p=1$ 时，闵可夫斯基距离即为曼哈顿距离 (Manhattan Distance)；

当 $p=2$ 时，闵可夫斯基距离即为欧式距离 (Euclidean Distance)；

当 $p \rightarrow \infty$ 时，闵可夫斯基距离即为切比雪夫距离 (Chebyshev Distance)。

不管闵氏距离的参数 p 如何变化，其衡量方法始终遵循距离越短，相似度越高这一准则。

2.2.2 余弦相似度

除了距离这一衡量因素外，一个向量空间中两个向量夹角间的余弦值也可以衡量两个个体之间差异的大小，余弦值接近 1，夹角趋于 0，表明两个向量相似；余弦值接近于 0，夹角趋于 90 度，表明两个向量不相似。余弦相似度的计算方法由下式给出：

$$k(x, y) = \frac{x * y^T}{||x|| * ||y||} \quad (2.3)$$

2.2.3 正交标准化

不论是距离度量还是余弦相似度度量，从公式（2.2）和（2.3）可以看出，两者都将不同维度的坐标的量纲看作一致来进行计算，因此使用这些公式前需要对坐标进行正交化处理。

以欧氏距离为例，标准化欧氏距离 (Standardized Euclidean Distance) 的思路为：既然数据各维分量的分布不一样，先将各个分量都“标准化”到均值、方差相等。假设样本集 X 的均值 (mean) 为 m ，标准差 (standard deviation) 为 s ， X 的“标准化变量”表示为：

$$X^* = \frac{X - m}{s} \quad (2.4)$$

由公式（2.4）可得标准化欧氏距离的公式为：

$$d_{12} = \sqrt{\sum_{k=1}^n \left(\frac{x_{1k} - x_{2k}}{s_k} \right)^2} \quad (2.5)$$

在适用场景上，欧氏距离能够体现个体数值特征的绝对差异，所以更多的用于需要从维度的数值大小中体现差异的分析，比如使用用户行为作为指标分析用户价值的相似情况（比较不同用户的消费能力），这属于价值度量；而余弦相似度对绝对数值不敏感，更多的用于使用用户对内容的评分来分析用户兴趣的相似程度（比如用户是否喜欢某商品），这属于定性度量。

2.3 性能度量

给定 n 个对象集合 $S = \{O_1, O_2, \dots, O_n\}$, 假设 $U = \{u_1, \dots, u_R\}$ 和 $V = \{v_1, \dots, v_C\}$ 表示 S 的两个不同划分并且满足 $\bigcup_{i=1}^R u_i = S = \bigcap_{j=1}^C v_j$, $u_i \cap u_{i^*} = \emptyset = v_j \cap v_{j^*}$, 其中有 $1 \leq i \neq i^* \leq R$, $1 \leq j \neq j^* \leq C$ 成立[22]。

假设 U 是外部评价标准 (true label), 而 V 是聚类结果。设定四个统计量:

a 为在 U 中为同一类且在 V 中也为同一类别的数据点对数 (TP);

b 为在 U 中为同一类但在 V 中却隶属于不同别的数据点对数 (FN);

c 为在 U 中不在同一类但在 V 中为同一类别的数据点对数 (FP);

d 为在 U 中不在同一类且在 V 中也不属于同一类别的数据点对数 (TN)。

其中, TP(True Positive) 是具有相同类标签并属于同一簇的点对的数量; FN(False Negative) 是具有相同类标签但分配给不同簇的点对的数量; FP (False Positive) 是具有不同类标签但被分配到同一聚类的点对的数量; TN(True Negative) 是具有不同类标签且被分配给不同簇的点对的数量。由于以上四种情况包含了数据点对的所有情况且没有重复, 故有 $a + b + c + d = C_n^2$ 成立。

2.3.1 外部指标

2.3.1.1 兰德系数 (RI)

兰德系数 (Rand Index)[23] 常用来衡量两个簇类的相似度:

表 2.1 四个统计量间的关系

	同类	不同类	总和 U
同类	a	b	a+b
不同类	c	d	c+d
总和 V	a+c	b+d	a+b+c+d

结合上表可知 RI 的计算方法如下:

$$RI = \frac{a + d}{a + b + c + d} \quad (2.6)$$

RI 系数的缺点是随着聚类数的增加，随机分配簇类向量的 RI 也逐渐增加。这是不合理的，因为随机分配簇类标记向量的 RI 应为 0。

2.3.1.2 调整兰德系数 (ARI)

调整兰德系数 (Adjusted Rand Index) 解决了 RI 不能很好的描述随机分配簇类标记向量的相似度问题。ARI 的定义：

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]} \quad (2.7)$$

其中 E 表示期望，max 表示取最大值。

ARI 的优点是对任意数量的聚类中心和样本数，随机聚类的 ARI 都非常接近于 0，取值在 [-1,1] 之间，负数代表结果不好，越接近于 1 越好，可用于聚类算法之间的比较。缺点则是 ARI 需要真实标签，在真实数据集中不好获取，在人工数据集中还需要人为添加标签，费时费力。

2.3.1.3 Fowlkes-Mallows 指数 (FMI)

Fowlkes-Mallows 指数 (FMI)[24] 定义为对精度 (分组点对的准确性) 和召回率 (正确分组在一起的点对的完整性) 的几何平均值：

$$FMI = \frac{a}{\sqrt{(a+c)(a+b)}} \quad (2.8)$$

FMI 取值范围为 [0,1]，其中 0 表示聚类结果与真实标签不相关，1 表示完全相关。

FMI 的优点:

1. 同时考虑准确率和召回率，提供一个平衡的聚类性能视图。
2. 结果在 0 到 1 之间。
3. 对于任意数量的样本和簇，随机 (均匀) 标签分配的 FMI 得分接近于 0。
4. 不对簇结构做假设。

FMI 的缺点:

1. 它是基于对元素的分析，这可能无法捕捉到簇更广泛的结构特性，比如它们的形状或分布。
2. 当数据集高度不平衡 (即一个类主导数据集) 时，FMI 可能无法准确反映聚类的有效性。

2.3.2 内部指标

对于标签未知的数据集，此时前文中的外部指标不再生效，必须使用模型本身的内部指标去度量聚类性能。

2.3.2.1 轮廓系数

每个样本有对应的轮廓系数，轮廓系数由两个得分组成：

- a: 样本与同一簇类中的其他样本点的平均距离；
- b: 样本与距离最近簇类中所有样本点的平均距离。

每个样本的轮廓系数定义为：

$$S = \frac{b-a}{\max(a,b)} \quad (2.9)$$

一组数据集的轮廓系数等于该数据集中每一个样本轮廓系数的平均值。

该评价指标具备以下特点:

1. 轮廓系数处于 $[-1,1]$ 的范围内，-1 表示错误的聚类，1 表示高密度的聚类，0 附近表示重叠的聚类；
2. 当簇密度较高且分离较大时，聚类的轮廓系数亦越大；
3. 凸簇的轮廓系数比其他类型的簇高。

2.3.2.2 Caliniski-Harabaz 指数

Caliniski-Harabaz 指数，又称为 CH 指数，是一种用来评估聚类算法效果的指标。它基于聚类结果的离散程度和簇之间的分离程度来评价一个聚类的质量。在聚类分析中，算法的最终目标是将相似的数据点分配到同一个簇中，同时不同簇之间应有较大的差异 (这也是本文提出的 PHANTOM 算法第二步的核心目标)。也就是说，一个好的聚类模型应该做到同一簇类的数据集尽可能密集，不同簇类的数据集尽可能远离。

为此，我们定义簇类散度矩阵：

$$W_k = \sum_{q=1}^k \sum_{x \in C_q} (x - c_q)(x - c_q)^T \quad (2.10)$$

簇间散度矩阵：

$$B_k = \sum_q n_q (c_q - c)(c_q - c)^T \quad (2.11)$$

其中 C_q 为簇类 q 的样本集， c_q 为簇类 q 的中心， n_q 为簇类 q 的样本数， c 为所有数据集的中心。

根据协方差的相关概念，我们用簇类散度矩阵的迹表示同一簇类的密集程度，迹越小，同一簇类的数据集越密集（方差越小）；簇间散度矩阵的迹表示不同簇间的远离程度，迹越大，不同簇间的远离程度越大（方差越大）。

结合评价聚类模型的标准，定义 Calinski-Harabaz 指数：

$$S(k) = \frac{\text{Tr}(B_k) * (N - k)}{\text{Tr}(W_k) * (k - 1)} \quad (2.12)$$

其中 N 为数据集样本数， k 为簇类个数。

该评价指标具备以下特点：

1. 当簇类密集且簇间分离较好时，Caliniski-Harabaz 分数越高，聚类性能越好；
2. 计算速度快；
3. 凸簇的 Caliniski-Harabaz 指数比其他类型的簇高。

需要注意的是，CH 指数只能作为聚类效果的初步评估，具体的分析还需要结合其他指标和具体问题而定。

3 PHANTOM 算法

为了实现任意形状快速聚类的目标，本文提出了 PHANTOM 算法，其步骤如下：（1）形状保持采样；（2）k 近邻代表点调整；（3）基于类间最小距离的凝聚聚类。

本章节所用到的参数如下表所示：

表 3.1 参数列表

参数	含义
D	原始数据集
N_c	最终要得到的聚类个数
N_r	形状保持采样要获得的代表点个数
k	代表点位置调整时的 k 近邻参数
N_{iter}	代表点位置调整时的迭代次数
M	去除离群点后的原始数据集
A_t	形状保持采样过程中的代表点合集
B_t	形状保持采样过程中的非代表点合集
X_{means}	数据集 M 的样本中心
d	数据集维度
F	吸引力系数矩阵
X	代表点坐标矩阵

PHANTOM 算法的伪代码如下：

输入：数据集 D ，聚类个数 N_c ，代表点个数 N_r ，k 近邻参数 k ，迭代次数 N_{iter} 。

输出：数据集 D 中各数据样本类别。

1. 对数据集进行形状保持采样，得到均匀分布于数据集集中的 N_r 个代表点；
2. 通过迭代调整的方法在 k 近邻代表点的共同作用下移动每个代表点的位置，直到迭代次数达到 N_{iter} ；
3. 通过类间最小距离的凝聚聚类将代表点合并，直至剩下 N_c 个数据簇，最终根据此结果对数据集进行聚类。

算法 1: PHANTOM(D, N_c, N_r, k, N_{iter})

3.1 形状保持采样

本文使用的形状保持采样具体如下：

1. 去除数据集 D 中的离群点；
2. 通过采样获得均匀连续的数据点（也就是 N_r 个采样点）。

对于去除离群点的必要性，本文认为离群点的存在会干扰采样点的选取。一方面，位于不同聚类之间的离群点在被选为采样点时会缩短类间距离，可能造成聚类划分的误判；另一方面，远离其他数据点的离群点在本文的算法下容易被单独分类为一个聚类，而这显然是无意义的。因此，本文的算法必须先将离群点处理掉。本文参考了不少已有的离群点算法[14,25,26]，最终决定采用 Bay 等人[14]提出的快速离群点检测算法。

在得到处理完离群点的数据集 D 后，采样工作就可以开始了。在这一部分中本文采取了迭代取样的思路。首先，我们用数据集 M 指代去除所有离群点后的数据集 D 。在选取第 t 个代表点时，我们设已被选取的代表点集合为 A_t ，未被选取的代表点集合为 B_t ，则应有 $B_t = (M \setminus A_t)$ 。在采样过程中每一个代表点只能从 B_t 中选取，且应有 $A_t \cup B_t = M$ ， $A_t \cap B_t = \emptyset$ 。在此基础上，我们按如下情况分类选取第 t 个代表点：

首先，如果 $t = 1$ ，即将要选取第一个代表点时，由于没有其他点作为参照，我们先计算数据集 M 的样本中心 $X_{means} = (x_{mean,1}, x_{mean,2}, x_{mean,3}, \dots, x_{mean,d})$ ，其中 d 为数据集维度。对于 $1 \leq i \leq d$ ， $x_{mean,i} = 1/2(\max_{1 \leq j \leq |M|} x_{j,i} + \min_{1 \leq j \leq |M|} x_{j,i})$ ，即通过求取数据集 M 中每一维度的极值两端的均值来寻找 M 的中心。得到中心 X_{means} 后，我们通过计算选取与之最近的数据点作为第一个代表点。

接着，对于 $1 \leq t \leq N_r$ ，即选取之后的代表点时，对于在此之前的 $(t-1)$ 个代表点组成的集合 A_t ，我们计算 B_t 中的每一个点 p_i 到其的最近距离，这个距离记为 $\min_{p_j \in A_t} d(p_i, p_j)$ ，由此我们可以将每组距离最近数据点记为 $(p_i, \arg \min_{p_j \in A_t} d(p_i, p_j))$ 。

在这些数据点对中，我们取出距离最大的点对，并使其中的 p_i 作为第 t 个代表点从 B_t 挪到 A_t 。用公式来描述上述过程即为下式：

$$a_t = \arg \max_{p_i \in A_t} d(p_i, \arg \min_{p_j \in A_t} d(p_i, p_j)) \quad (3.1)$$

其中 $1 \leq t \leq N_r$ 。

最后，当 $t > N_r$ 时结束选取过程。

以图3.1为例，对于数据点 P_1 至 P_9 ，前两次迭代依次选取了 P_5 和 P_3 作为代表点。在第三次迭代前， $A_t = \{P_3, P_5\}$ ， $B_t = \{P_1, P_2, P_4, P_6, P_7, P_8, P_9\}$ ，此时 B_t 中各点到 A_t 的最短距离的合集为 $\{(P_1, P_3), (P_2, P_3), (P_4, P_5), (P_6, P_5), (P_7, P_5), (P_8, P_5), (P_9, P_5)\}$ ，经比较可知在这之中距离最大的是 $d(P_7, P_5)$ ，因此将 P_7 作为第三个代表点加入 A_t ，同时将其移出 B_t 。

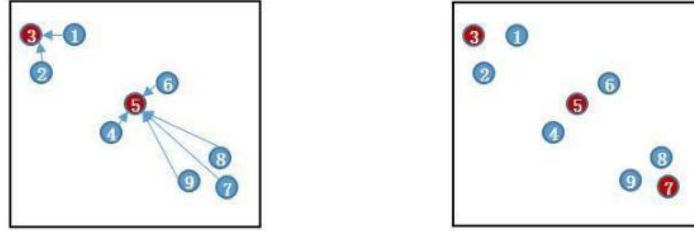


图 3.1 迭代前后

形状保持采样的伪代码如下：

输入： 去除离群点后得到的数据集 M ，数据集中心 X_{means} ，代表点个数 N_r

输出： 包含 N_r 个代表点的合集 A_t

初始化： $A_t = \emptyset, t = 0$

while $t < N_r$ **do**

$t++$;

if $t = 1$ **then**

$add \arg \min d(p_i, X_{means})$ to A_t ;

$sub \arg \min_{\substack{p_i \in B_t \\ p_i \in B_t}} d(p_i, X_{means})$ to B_t ;

else

$add \arg \max_{p_i \in A_t} d(p_i, \arg \min_{p_j \in A_t} d(p_i, p_j))$ to A_t ;

$sub \arg \max_{\substack{p_i \in A_t \\ p_j \in A_t}} d(p_i, \arg \min_{p_j \in A_t} d(p_i, p_j))$ to B_t ;

end

end

算法 2: 形状保持采样 (M, X_{means}, N_r)

在上述的取样过程中，代表点在保持彼此距离最大化的同时尽可能的覆盖到了更多的数据点范围，最大程度地保留了数据集原本的形状（这也是形状保持采样算法名称的由来）。

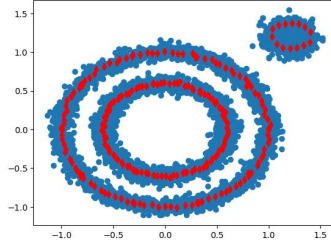


图 3.2 形状保持采样

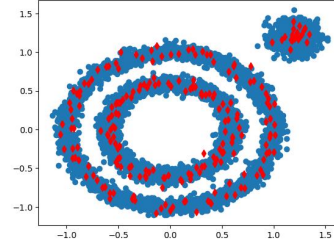


图 3.3 随机取样

从图 (3.2)和图 (3.3)的对比中可以看出，形状保持采样对数据集的采样更为均匀，而随机采样除了密度不均以外还容易混淆不同聚类间的边界，进而导致数据集原本的结构被破坏。综上所述，形状保持采样的性能要明显更好。

3.2 代表点位置调整

经过了形状保持采样的选取，我们得到了含有 N_r 个代表点的数据集 A_t 。然而，此时的数据集并不能直接用于聚类，因为此时同一聚类内部的代表点仍然保持着相对距离最远的规则，类内代表点和类间代表点缺乏区分度。为此，我们有必要对各个代表点进行调整，使类内代表点相互靠拢，同时类间代表点相互远离，从而使聚类结构更加清晰。为了达成这一目标，本文决定让每个代表点的坐标在其 k 近邻代表点的影响下不断向 k 近邻代表点的中心移动，直至达到迭代次数 N_{iter} 。对于 A_t 中的每一个代表点 a_i ，将其未经过调整前的坐标记为 $X_{i,0}$ ，则在经过 t 次调整 ($1 \leq t \leq N_{iter}$) 后的坐标 $X_{i,t}$ 可由下面的递推公式得出：

$$X_{i,t} = (1 - \alpha)X_{i,t-1} + \alpha \frac{\sum_{a_j \in N_k(i)} \frac{X_{j,t-1}}{\|X_{i,0} - X_{j,0}\|}}{\sum_{a_j \in N_k(i)} \frac{1}{\|X_{i,0} - X_{j,0}\|}} \quad (3.2)$$

其中 $X_{i,t-1}$ 代表上一次调整得到的坐标， $N_k(i)$ 代表 a_i 的 k 近邻代表点合集， α 是取值介于 (0,1) 的影响力参数， α 越大，每一次调整对 a_i 的影响就越大。然而，过大的 α 可能会扭曲聚类结构，因此 α 的取值范围一般应该位于 (0, 0.3) 之间。而对于部分对调整幅度有要求的数据集，则应考虑在降低 α 的同时增加 N_{iter} ，以期得到更为平滑的迭代过程和结果。在公式 (3.2) 中， $X_{i,t}$ 在每一次迭代中逐渐向着 $N_k(i)$ 的中心靠拢。特别是当 k 值较小时， a_i 的 k 近邻代表点合集和 a_i 位于同一个聚类的可能性很高，这也就

意味着上式让同一聚类的代表点相互靠近，同时使得不同聚类的代表点因此而相互远离。这时，利用局部连通性原理就能够很好地判别同一聚类的代表点。

对于公式 (3.2)，将单个代表点扩展到整体，则可以用如下公式得到第 t 次迭的代表点整体坐标的计算方法：

$$X_t = FX_{t-1} \quad (3.3)$$

其中 X_t 代表第 t 次迭代得到的代表点坐标矩阵， F 则表示第 t 次迭代的过程中的系数矩阵， $F_{i,j}$ 表示 a_i 对 a_j 的吸引力系数。 $F_{i,j}$ 和 X_t 的计算方法分别由 3.4 和 3.5 给出，其中 $\sum_{j=1}^{N_r} F_{i,j} = 1$ 。

$$F = \begin{cases} (1 - \alpha) & , \quad \text{if } i = j \\ \alpha \frac{\frac{1}{\|X_{i,0} - X_{j,0}\|}}{\sum_{a_j \in N_k(i)} \frac{1}{\|X_{i,0} - X_{j,0}\|}} & , \quad \text{if } a_j \in N_k \\ 0 & , \quad \text{otherwise} \end{cases} \quad (3.4)$$

$$X_t = (X_{1,t}, X_{2,t}, \dots, X_{N_r,t})^T \quad (3.5)$$

3.3 基于类间最小距离的凝聚聚类

经过了前文的一系列处理，代表点合集的聚类结构逐渐清晰，因此最后一步要做的便是将代表点分类为给定个数 N_c 个不同的集合，并以此为标准将原数据集 D 中的所有点进行分类。为了实现这一目的，本文采用了凝聚聚类算法对数据簇进行合并。

如 2.1.1 所述，凝聚聚类有多种合并准则。事实上，在文中提到的若干方法中，基于类间最小距离的判别方法速度最快，但在处理一般聚类问题时因其判别方式过于粗糙（只有非常少的数据点参与了单次聚类过程）而导致效果不佳。但是，考虑到目前需要聚类的代表点合集已经经过了调整，类内数据点相对集中而类间数据点相对分散，因而使用类间最小距离已经足够对代表点进行正确分类了，能够最大程度的节约时间和计算资源。因此，本文在经过多次实验验证后选择用基于类间最小距离的凝聚聚类来对代表点进行分类。

基于类间最小距离的凝聚聚类的具体步骤如下：

1. 将每个代表点看作是单独的数据簇；
2. 用每对数据簇中距离最近的一对点的距离代表数据簇间的距离；
3. 寻找距离最近的数据簇并将其合并；
4. 重复步骤 2，3 直到数据簇的数量降到 N_c 。

例如，对于数据簇 A 和 B ，它们的相对距离的计算方法可由下式给出：

$$dist_{min}(A, B) = \min_{a_i, b_j} \{|a_i - b_j|\} \quad (3.6)$$

其中 $a_i \in A$, $b_j \in B$ 。

在上述步骤全部完成后，将各代表点的标签赋给其所代表的原始数据集样本。最后，对于在形状保持采样的第一步中去除的离群点，我们按照就近原则，通过计算其到每一个类的所有点的平均距离来寻找和它最近的类并将之划分进去。这样，本文介绍的算法就顺利地完成了对原始数据集的划分。

3.4 算法复杂度分析

本文提出的 PHANTOM 算法由形状保持采样，代表点位置调整和凝聚聚类三部分组成。这三个部分彼此独立，依序进行，因此本文算法的复杂度可以对其分别计算。

首先，对于形状保持采样部分，第一步的离群点排除方法是近似线性的，时间复杂度约为 $O(N)$ ，之后的均匀采样在每一个代表点的选取过程中都要遍历一次所有数据点来寻找非代表点距离代表点最小距离的最大值，因此这部分的时间复杂度为 $O(N_r \cdot N)$ ，其中 N_r 为代表点个数， N 为原始数据集样本总数。

接着，对于代表点位置调整部分，算法的时间复杂度主要受迭代次数 N_{iter} ， k 近邻参数 k 以及代表点总数 N_r 影响。在每一次迭代中，由公式 3.3，3.4 和 3.5 结合可知每一个代表点要分别被其 k 近邻的每一个代表点的坐标影响，即需要计算 k 次，同时 k 近邻算法本身的时间复杂度为 $O(N_r)$ ，因此综合来看，代表点位置调整部分的时间复杂度为 $O(N_r \cdot k \cdot N_{iter})$ 。

最后，对于凝聚聚类部分而言，大部分主流方法的时间复杂度都有 $O(N^3)$ 。但是就如 3.3 章所说，本文所用的基于类间最小距离的凝聚聚类时间复杂度较低，所以可借助最大堆快速找到类间距离最近的数据点。因此，这一部分的复杂度为 $O(N_r^2 \cdot \log N_r)$ 。

综合来说，由于 k 近邻参数 k 和迭代次数 N_{iter} 都是人为选取的常数，并且一般远小于样本数，因此可知 $N_r \cdot k \cdot N_{iter} \ll N_r \cdot N$ 。所以，本文提出的 PHANTOM 算法的时间复杂度应为 $O(N_r \cdot N + N_r^2 \cdot \log N_r)$ 。

3.5 本章小结

在本章中，本文介绍了 PHANTOM 算法的具体实现步骤。首先，PHANTOM 算法去除了原始数据集中的离群点，通过形状保持采样尽可能均匀地采集到了原始数据集中的代表点，并使其它它们之间的相对距离尽可能的远；接着，为了解决代表点在类内和类间缺乏区分度的问题，PHANTOM 算法利用每个代表点的 k 近邻代表点调整位置，使得类内代表点的距离缩短而类间代表点的距离增大；最后，对于已经完成预处理的代表点合集，PHANTOM 算法使用基于类间最小距离的凝聚聚类来完成最后的分类工作。本文的算法时间复杂度为 $O(N_r \cdot N + N_r^2 \cdot \log N_r)$ ，总体要优于目前主流的任何形状聚类方法。

4 实验分析

在本章中，本文将用实验证明PHANTOM算法的可行性。其中，4.1节介绍了本章实验中使用的数据集；4.2节说明了PHANTOM算法在实验中的具体参数选取及其依据；4.3节在人工数据集上展示了PHANTOM算法的聚类能力；4.4节和4.5节横向对比了其他主流的任意形状聚类算法和本文算法在性能上和可拓展性上的差异，最终证明本文提出的算法拥有更好的性能。

4.1 实验设置

表 4.1 人工数据集

数据集	名称	样本个数	数据集维度	聚类个数
DS1.1	cth3	1146	2	4
DS1.2	d6	1400	2	4
DS1.3	ls3	1735	2	6

表 4.2 UCI 数据集

数据集	名称	样本个数	数据集维度	聚类个数
DS2.1	Iris	150	4	3
DS2.2	Wine	178	13	3
DS2.3	Vehicl	752	18	4
DS2.4	Optdigits	3823	64	10

表 4.1列出了部分常用的人工数据集，表 4.2列出了部分常用的 UCI 数据集。此外，为了进一步分析算法的可拓展性，本文在数据生成工具 SynDECA 的帮助下生成了数据集 DS 3.1 至 DS 3.7 以及 DS 4.1 至 DS 4.7。其中数据集 DS 3.1 至 DS 3.7 控制样本的维度保持为 8，使样本数量以 5000 为间隔逐渐增加，范围为 5000 到 35000；而数据集 DS 4.1 至 DS 4.7 控制样本数量保持为 8000，使样本维度以 8 为间隔逐渐增加，维度范围为 8 到 56。

本文全部实验运行于装配 16G 内存和 2.6GHz 处理器、运行 Win10 操作系统台式电脑上。全部算法均采用 Matlab 实现。算法聚类结果准确性用标准互信息 NMI 衡量。

4.2 参数选择

由算法 1 给出的伪代码可知，本文的 PHANTOM 算法需要事先输入聚类个数 N_c ，代表点个数 N_r ， k 近邻参数 k 和迭代次数 N_{iter} 。由于本文使用的数据集都是在已知聚类个数的前提下进行性能测试(这也是本文算法的不足之处)，故需要考虑的参数仅有 N_r ， k 和 N_{iter} 。

4.2.1 代表点个数

在 3.4 节中，本文经论证得出 PHANTOM 算法的时间复杂度为 $O(N_r \cdot N + N_r^2 \cdot \log N_r)$ 。可以发现，对于给定数据集 D 来说，数据集的样本个数 N 是常数，因此算法的时间复杂度仅受 N_r 影响。因此， N_r 的取值对于本文算法而言至关重要。一方面，过低的 N_r 会使取点不均，影响算法的结果和性能；另一方面，过高的 N_r 会造成时间和计算资源的浪费，同时也会使代表点本身的代表性下降。因此，对 N_r 的选取应当结合实验结果敲定。

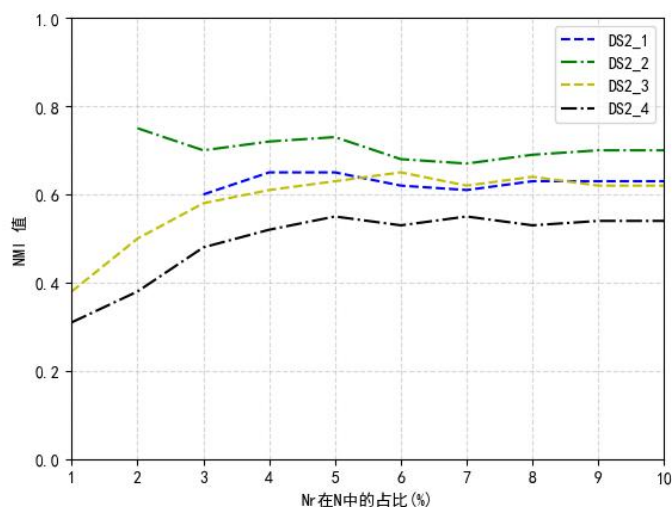


图 4.1 N_r 的值对聚类结果的影响

如图 4.1 所示，对于数据集 DS2.1 到 DS2.4，随着 N_r 在 N 中的占比不断提升，PHANTOM 算法的性能整体上是逐渐上升直到趋于稳定的，特别是当 N_r 在 N 中的占比提升至 0.05 以后，各个数据集的分类结果准确性提升受比例增加的影响可以忽略不计，因而本文最终选取 $0.05N$ 作为 N_r 的取值。

4.2.2 k 近邻参数

正如在 3.2 节中所提及的，在代表点调整的过程中，较小的 k 可以使被调整点及其 k 近邻代表点在同一类内的可能性增加，但这也不是说 k 的值越小越好，因为过

小的 k 值同样可能会扭曲聚类结构，使得代表点的分布变为更细碎的数据簇，从而阻碍后续的聚类。因此，本文在数据集 DS2.1 到 DS2.4 上控制其他变量恒定的同时仅改变 k ，得到的结果如下图所示：

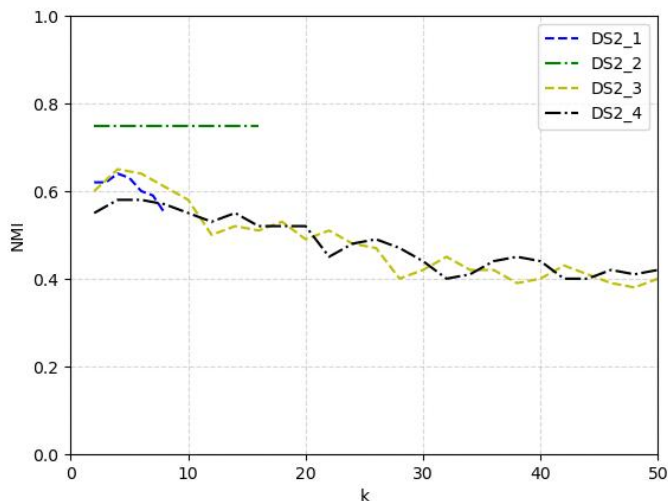


图 4.2 k 的值对聚类结果的影响

可以看到，在不同的 k 值下，尽管有的数据集不怎么受其影响，但大部分数据集的结果准确性在 k 变大的过程中都如分析所预想的一样呈现出先增后减的趋势。从整体上看，当 $k=4$ 时四个数据集的聚类效果都相对比较好，因此本文将 k 固定为4。

4.2.3 迭代次数

对于代表点调整部分，调整的目的是为了后续更好地分类，而当迭代次数达到一定值的时候，代表点之间已有足够的区分度，此时继续迭代并不能改善最后分类结果的准确性，反而会造成时间和计算资源的浪费。因此，应该选取一个适中的 N_{iter} 值。

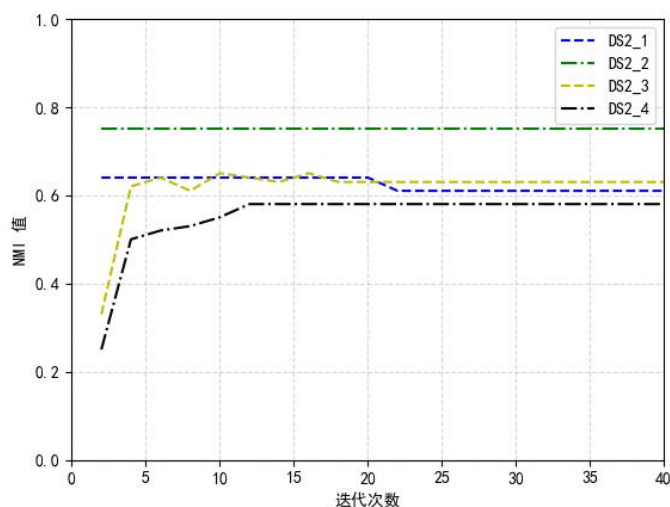


图 4.3 N_{iter} 的值对聚类结果的影响

可以看到，在 $N_{iter} < 20$ 时，各个数据集的分类准确性仍然会因 N_{iter} 的增长而发生波动，但当 $N_{iter} \geq 20$ 后，各个数据集的分类准确性不再增加，波动较小，因此可以认定 $N_{iter}=20$ 是合适的。

4.3 聚类能力验证

图 4.4，4.5，4.6给出了 PHANTOM 算法在三个人工数据集上的聚类结果，这表明 PHANTOM 算法具有识别任意形状聚类的能力。

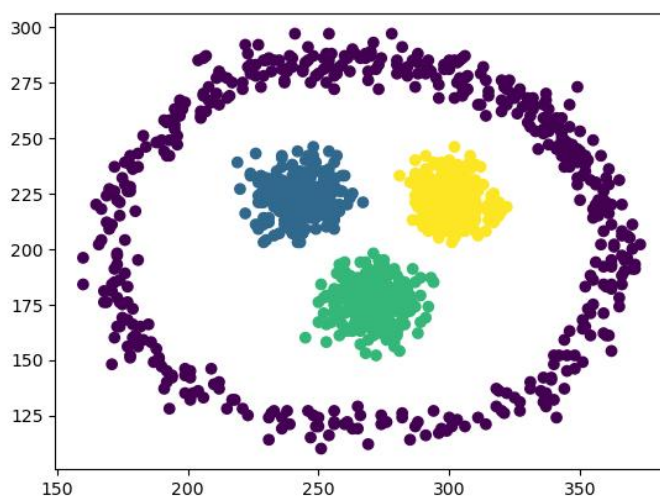


图 4.4 DS 1.1 数据集

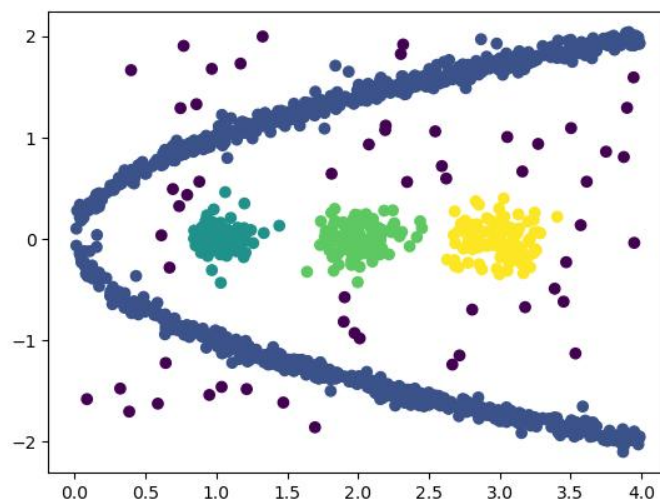


图 4.5 DS 1.2 数据集

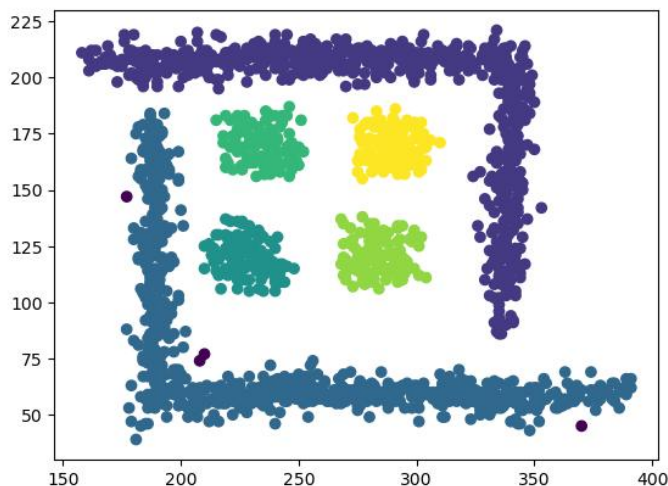


图 4.6 DS 1.3 数据集

4.4 算法性能对比

为了验证本文所述方法相较于目前主流的任何形状聚类算法是否具有优势，本节挑选了层次聚类的主流算法 CHAMELEON，基于密度的聚类方法 DBSCAN 和基于采样的方法 ABACUS 在数据集 DS2.1 至 2.4 上与本文提出的算法进行比较。这些方法都具有任意形状聚类的能力，为了公平起见，所有的方法都选取了最适宜的参数。各算法的准确性和用时的对比结果如下所示：

表 4.3 各算法在 UCI 数据集上的准确性对比 (NMI)

数据集	CHAMELEON	DBSCAN	ABACUS	PHANTOM
DS2.1	0.61	0.63	0.64	0.66
DS2.2	0.68	0.65	0.64	0.75
DS2.3	0.64	0.61	0.59	0.65
DS2.4	0.51	0.53	0.51	0.57

表 4.4 各算法在 UCI 数据集上的运行时间对比 (秒)

数据集	CHAMELEON	DBSCAN	ABACUS	PHANTOM
DS2.1	0.02	0.01	0.02	0.02
DS2.2	0.16	0.18	0.22	0.12
DS2.3	0.87	0.69	0.82	0.43
DS2.4	3.41	2.83	3.04	1.28

综合来看，本文提出的 PHANTOM 算法在准确性上要优于上文选取的其他几种方法。而在用时上，本文提出的算法在较大规模的数据集上优势明显，在小规模数据集上反而会耗时更长。

4.5 算法可拓展性对比

为了验证本文提出的 PHANTOM 算法在样本数量提升时的表现，本文基于数据集 DS3.1 至 3.7 对不同算法的运行时间进行了对比，其中数据集 DS3.1 至 3.7 的数据维数保持一致，样本数量以 5000 为间隔变化。得到的结果如图4.7所示：

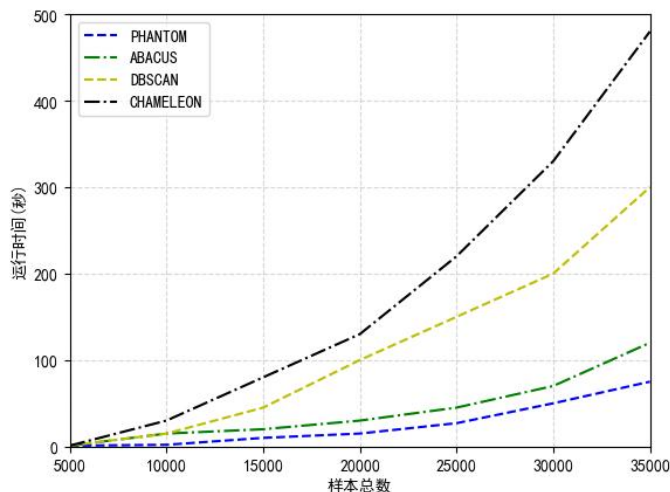


图 4.7 各算法运行时间随样本数量的变化

另一方面，为了验证本文提出的PHANTOM 算法在样本维数提升时的表现，本文基于数据集 DS4.1 至 4.7 对不同算法的运行时间进行了对比，其中数据集 DS4.1至 4.7 的样本数量保持一致，数据维数以 8 为间隔变化。得到的结果如图 4.8所示：

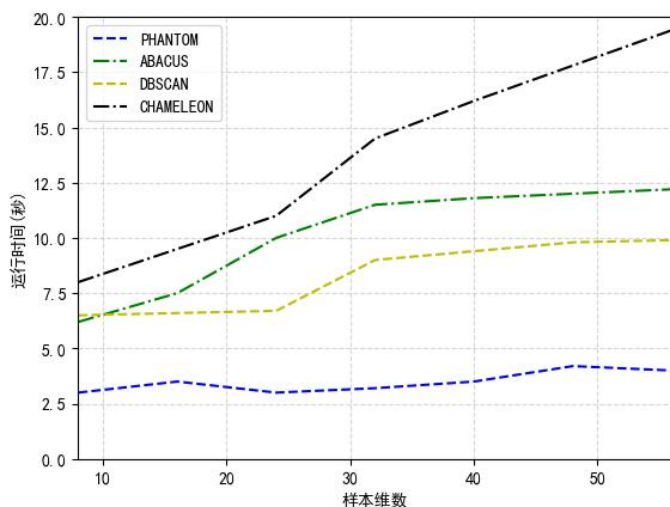


图 4.8 各算法运行时间随样本维数的变化

从图 4.7可以看出，随着样本数量的提升，各算法的运行时间在总体上都呈上升趋势，在这之中，PHANTOM 算法不仅在数据集 DS3.1 至 3.7 上的用时都是最短，而且上升趋势也较其他算法更平缓。在这之中，ABCUS 算法与本文提出的 PHANTOM 算法上升趋势相近，但 PHANTOM 算法始终保持更快的运行速度。

从图 4.8可以看出，随着样本维数的提升，其他算法的运行时间都有明显的增加，而 PHANTOM 算法的运行时间上升缓慢。综合来看，本文提出的 PHANTOM 算法的可拓展性更好。

5 总结

任意形状快速聚类算法种类繁多，本文在调研了层次聚类、基于图的聚类、基于密度的聚类、基于划分的聚类以及基于采样的聚类后提出了一种基于形状保持采样的任意形状聚类算法 PHANTOM。该算法分为以下几个步骤进行：

形状保持采样： PHANTOM 算法去除了原始数据集中的离群点，通过形状保持采样尽可能均匀地采集到了原始数据集中的代表点，并使其它它们之间的相对距离尽可能的远；

代表点位置调整： 为了解决代表点在类内和类间缺乏区分度的问题， PHANTOM 算法利用每个代表点的 k 近邻代表点调整位置，使得类内代表点的距离缩短而类间代表点的距离增大；

基于类间最小距离的凝聚聚类： 对于已经完成预处理的代表点合集， PHANTOM 算法使用基于类间最小距离的凝聚聚类来完成最后的分类工作。经过实验论证，该算法的表现要优于目前主流的任意形状聚类方法，性能和可拓展性良好。

尽管如此，该算法在现阶段仍有很多缺点亟待解决。首先，该算法依赖于用户输入的聚类个数，只有在对数据类别和规模有一定了解时才可以使用；其次，该算法的部分参数选取依赖于实验的反馈，其中最优位置的选取缺乏理论支撑。希望在不久的将来能够解决这些问题，进一步巩固和完善本文的算法；最后，该算法目前仅适用于静态数据集，但实践中常常会出现数据集的增量更新，如何通过修改本文算法来更加适应市场需求也是未来需要解决的问题。

参考资料

- [1] Hao Huang, Yunjun Gao, Kevin Chiew Lei Chen, Qinming He. Towards effective and efficient mining of arbitrary shaped clusters[C], in: ICDE 2014, 2014, pp. 28-39.
- [2] Marijus Bernotas, Kazys Karklius , Remigijus Laurutis, Asta Slotkienė. The peculiarities of the text document representation, using ontology and tagging-based clustering technique[J], in: INF TECHNOL CONTROL, 2007, 36(2).
- [3] Zhihao Wang, Jian Chen, Steven C. H. Hoi. Deep learning for image super-resolution: a survey[J], in: IEEE/TPAMI , 2021, 43(10), pp. 3365-3387.
- [4] Narumol Chumuang, Mahasak Ketcham. Model for Handwritten Recognition Based on Artificial Intelligence[C], in: iSAI-NLP, 2018, pp. 1-5.
- [5] Andre Esteva, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau, Sebastian Thrun. Correction: Corrigendum: Dermatologist-level classification of skin cancer with deep neural networks[C], in: Nature, 2017, 546(686)..
- [6] Rui Xu, Donald C. Wunsch. Clustering algorithms in biomedical research: a review[J], in: IEEE/RBME, 2010, volume 3, pp. 120-154.
- [7] Girish Punj, David W. Stewart. Cluster analysis in marketing research: review and suggestions for application[J], in: Journal of marketing research, 1983, 20(2), pp. 134-148.
- [8] Girish Prayag, Marta Disegna, Scott Allen Cohen, Hongliang (Gordon) Yan. Segmenting markets by bagged clustering: Young Chinese travelers to Western Europe[J], in: Journal of Travel Research, 2015, 54(2), pp. 234-250.

- [9] Negar Ohadi, Ali Kamandi, Mahmood Shabankhah, Seyed Mohsen Fatemi, Seyed Mohsen Hosseini, Alireza Mahmoudi. SW-DBSCAN: A grid-based DBSCAN algorithm for large datasets[C], in: ICWR 2020, 2020, pp. 139-145.
- [10] P. Viswanath, R. Pinkesh. l-DBSCAN: A fast hybrid density based clustering method[C], in: ICPR'06, volume 1, 2006, pp. 912-915.
- [11] 周志华. 机器学习[M]. 北京. 清华大学出版社. 2016. 214
- [12] Vineet Chaoji, Mohammad Al Hasan, Saeed Salem, Mohammed J. Zaki. SPARCL: Efficient and effective shape-based clustering[C], in: ICDM 2008, 2008, pp. 93-102.
- [13] Vineet Chaoji, Geng Li, Hilmi Yildirim, Mohammed J. Zaki. ABACUS: Mining arbitrary shaped clusters from large datasets based on backbone identification[C], in: SIAM 2011, 2011, pp. 295-306.
- [14] Stephen D. Bay, Mark Schwabacher. Mining distance-based outliers in near linear time with randomization and a simple pruning rule[C], in: KDD'03, 2003, pp. 29-38.
- [15] Sudipto Guha, Rajeev Rastogi, Kyuseok Shim. CURE: An efficient clustering algorithm for large databases[C], in: SIGMOD, 1998, pp. 73-84.
- [16] Weiling Cai, Songcan Chen, Daoqiang Zhang. Fast and robust fuzzy c-means clustering algorithms incorporating local information for image segmentation[J], in: Pattern Recognition, 2007, 40(3), pp. 825-838.
- [17] Ulrike von Luxburg. A tutorial on spectral clustering[J], in: Statistics and Computing, 2007, 17(4), pp. 395-416.
- [18] Wen-Yen Chen, Yangqiu Song, Hongjie Bai, Chih-Jen Lin, Edward Y. Chang. Parallel spectral clustering in distributed systems[C], in: IEEE/TPAMI, 2011, 33(3), pp. 568-586.

- [19] Tiantian Zhang, Bin Wu, A method for local community detection by finding core nodes[C], in: 2012 IEEE/ACM, 2012, pp. 1171-1176.
- [20] Qinna Wang, Eric Fleury. Community detection with fuzzy community structure[C], in: 2011 IEEE/ASONAM, 2011, pp. 575-580.
- [21] G. Karypis, Eui-Hong Han, V. Kumar. Chameleon: Hierarchical clustering using dynamic modeling[J], in: Computer, 1999, 32(8), pp. 68-75.
- [22] Ka Yee Yeung, Walter L. Ruzzo. An empirical study on principal component analysis for clustering gene expression data[R], in: Technical Report UW-CSE-2000-11-03.
- [23] William M. Rand. Objective criteria for the evaluation of clustering methods[J], in: JASA, 2012, 66(336), pp. 846-850.
- [24] Hirschberg, Julia Bell, Rosenberg, Andrew. V-measure: A conditional entropy-based external cluster evaluation measure[C], in: EMNLP-CoNLL, 2007, pp. 410-420.
- [25] Fabrizio Angiulli, Clara Pizzuti. Fast outlier detection in high dimensional spaces[C], in: PKDD, 2002, volume 2431, pp. 15-27.
- [26] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, Jörg Sander. LOF: identifying density-based local outliers[C], in: SIGMOD, 2000, pp. 93-104.

致谢

时光飞逝，四年的大学生活转眼迎来了尾声。这四年里，有拼搏，有欢笑，有结伴而行时克服的困难，有独自前行时战胜的挑战。总的来说，我认为自己在这四年里没有虚度光阴，解决了很多自己最初觉得无法完成的难题，和朋友们创造了很多难以忘却的回忆。我很感谢武汉大学，它富有人文气息的学术环境令我受益匪浅，宽严适中的学习任务使我可以在完成学业的同时自由地去开拓自己的眼界，认识新的朋友，留下珍贵的回忆。

感谢我的导师黄浩教授对我一直以来的帮助。早在我大三时，黄老师就开始教授我统计学习的相关知识，这对我完成这篇论文产生了莫大的帮助。在这篇论文的选题到定稿的各个环节，黄老师在各个方面给出了宝贵的参考意见，我想表达自己由衷的感谢之情。

感谢武汉大学遥感信息工程学院的郭雨霄同学，他每天督促我和他一起写论文，在客观上促使这篇论文早日完成，我很感谢他帮助我治好了自己拖延症的毛病。

感谢武汉理工大学的奚兆阳同学，虽然他没有直接参与帮助我的论文书写工作，但我曾就本文方向与他讨论出不少结果，并因此受益良多，我想感谢他一直以来对我的鼓励和支持。

感谢我的父母和朋友，尽管我总是成熟地向他们寻求帮助，但他们总是愿意回应我、帮助我。我想，在未来我还会遇到很多帮助我的人，我想平等地感谢这之中的每一个人，也衷心希望我今日的努力可以让将来的自己也能帮到他们。